Day 7 Evidence Workbooklet

Ordered Validation and First-True Behavior

Engineering practice → ordered rules → first true branch → skipped branches → support plan

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/42		Mastery check	secure / developing / needs support	

The sensor-kit checkout table now uses several rules in a fixed order. A kit might have more than one issue, but the branch outcome should identify the first action the team should take. Today the focus is ordered validation: trace which condition is first true and explain why later branches are skipped.

Engineering task	Evaluate a sensor kit through an ordered checkout policy without losing evidence about skipped issues.
Computational move	Read an `if/elif/else` chain as an ordered list of checks where the first true branch wins.
Python focus	`if`, `elif`, `else`, first-true behavior, skipped branches, comparison and string-label checks.
Evidence to keep	rule order, condition result, first true branch, skipped later branch, branch outcome assigned, and rule-order risk.

Day 7 working rules

1. Python checks an `if/elif/else` chain from top to bottom.
2. The first true branch runs.
3. Later branches are skipped after the first true branch.
4. Rule order can change the branch outcome.
5. A reliable branch-decision note names the first true rule and any important skipped issue.

1 Read the ordered rules before tracing cases

recognize predict

The checkout desk now uses more than one rule. Python checks the rules in order. The first true rule assigns the branch outcome and the later rules are skipped.

```
kit_label = "S-22"
charge_percent = 76
calibration_label = "current"
borrower_initials = "MR"
damage_note = ""

if damage_note != "":
    action = "repair bench"
elif charge_percent < 80:
    action = "charge station"
elif calibration_label != "current":
    action = "calibration bench"
elif borrower_initials == "":
    action = "borrower log needed"
else:
    action = "release kit"
```

Pts.	Prompt	My evidence
1	Which rule is checked first?	
1	Which rule is checked second?	
1	Which rule is true for this kit?	
1	Which branch outcome is assigned?	

2 Trace first-true behavior

trace explain

Complete the table. Stop at the first true rule for each case.

Pts.	Damage?	Charge	Calibration	Initials	First true rule and branch outcome
2	no	76	current	MR	
2	cracked case	76	expired	MR	
2	no	91	expired	MR	
2	no	91	current	blank	

First-true reminder

If an early rule is true, Python does not continue looking for a later, also-true rule. Your evidence should name the first true rule, not just every possible problem.

3 Explain skipped branches

explain debug

A kit may have more than one problem. Ordered selection decides which problem is handled first.

```
charge_percent = 72
calibration_label = "expired"
damage_note = ""

if damage_note != "":
    action = "repair bench"
elif charge_percent < 80:
    action = "charge station"
elif calibration_label != "current":
    action = "calibration bench"
else:
    action = "release kit"
```

Pts.	Prompt	My response
2	Which branch outcome is assigned?	
2	Is the calibration rule also true for this kit? Explain.	
2	Why does the calibration branch not run?	
2	What would be dangerous about reporting only the final branch outcome without the skipped issue?	

4 Find a rule-order problem

debug test

A teammate proposes moving the release check above the damage check.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    action = "release kit"
elif damage_note != "":
    action = "repair bench"
else:
    action = "hold for review"
```

Pts.	Prompt	My response
2	What important condition is checked too late?	
2	Give a kit case that would be assigned unsafely by the proposed order.	
2	What branch outcome should that case receive?	
2	Write one sentence explaining the rule-order risk.	

5 Constrained edit of one rule

implement debug

Only edit the calibration rule below. Do not rewrite the whole decision chain.

```
    elif ________________________________________________:
        action = "calibration bench"
```

Pts.	Prompt	My response
3	Complete the calibration rule.	
2	Explain why the rule should compare to `"current"`.	
2	Give one calibration label that should produce the calibration bench.	
1	Name one later branch that would be skipped if this rule is true.	

6 Transfer and support plan

transfer explain

Pts.	Ordered-validation note
4	Write a note that explains how ordered validation differs from a single `if`/`else`. Use first true and skipped branch in your explanation.
2	Circle the support plan you would use next: rule order / skipped branch / comparison / string label / written explanation. Name one next action: _____

Bridge to Day 8

Day 8 uses expected-vs-actual evidence to debug a branch decision when the selected action does not match the rule intention.

VIP Lab Alignment

Bridge Day 7 • Contract 16b4e542994c

This page replaces the earlier wrapper-based lab bridge and follows the current top-level notebook interface.

Why this page is here

VIP Lab Day(s): 4 **Activities:** 18.13, 18.14, 18.15, 18.16, 18.17

Carry comparison and branch-order evidence into the current top-level decision cells; Activity 18.14 supplies its loop.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.13 `18-13-work / 18-13-transfer`	**Compare With Tolerance**	Compute absolute distance between two values.
18.14 `18-14-work / 18-14-transfer`	**Count High Readings**	Read a supplied loop without having to design it yet.
18.15 `18-15-work / 18-15-transfer`	**Lamination Fee**	Translate three quantity intervals into ordered branches.
18.16 `18-16-work / 18-16-transfer`	**Temperature Status**	Compare a measurement with a maximum.
18.17 `18-17-work / 18-17-transfer`	**Pickup Window**	Calculate round-trip travel time.

Readiness check before you code

1. Identify which activity supplies a loop and state exactly what decision you are responsible for writing inside it.

   _____

2. For one of Activities 18.13, 18.15, 18.16, or 18.17, name an equality boundary that must receive its own test.

   _____

Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those studio cells are scored for independent mastery on Lab Days 5–7.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Contract-controlled page • generated from the current VIP activity profiles and stable notebook cell IDs